University of Pisa

Parallel and Distributed Systems

Optimization of Partitioned Hash Join with Duplicates

Module 3 - Report

Gianluca Panzani

May 12, 2026

Contents

1 Implementation

1.1 Implementation

This module keeps the same partitioned hash-join pipeline used in the previous report: deterministic input generation, partitioning of both relations, independent local joins on equal partition ids, and final accumulation of `join_count`, `checksum1` and `checksum2`. The partition mapping is the same xorshift-style mixer used before:

$$key \mathrel{\hat{=}} key \gg 33, \quad key \mathrel{\hat{=}} key \gg 17, \quad key \mathrel{\hat{=}} key \gg 9, \quad pid = key\&(P-1).$$

The implementation work in this module is therefore focused on expressing the existing algorithm with OpenMP and on evaluating how OpenMP scheduling reacts to uniform and skewed partition workloads.

The repository now contains one sequential baseline and six OpenMP executables. The main OpenMP families are loop, explicit-task and taskloop; the source files are the corresponding `hashjoin_omp_*` files, plus the `_wb` versions that add workload-aware join ordering. All versions use the same flat open-addressing count table in the local build/probe phase, so the comparison is mostly about the parallel execution model and scheduling policy, not about a different join algorithm.

The evaluation is based on a grid-search workflow. Instead of choosing one OpenMP configuration manually, each family is first executed on a larger grid of thread counts, schedules, chunks, block sizes and task grains. The resulting CSV files are then analyzed to identify stable regions, not only isolated minimum points. The final plots and tables use a reduced grid derived from this first exploration, so the reported results remain representative while avoiding an unnecessarily large benchmark campaign.

1.2 Parameters

The benchmark scripts run Slurm jobs on one node of the `spmcluster` and append one CSV row per configuration in `src/results/`. The fixed parameters of the main campaign are $N_R = N_S = 50\,000\,000$, $P = 256$, `seed=13`, `max_key=1 000 000` and block size 32768 in the reduced grid.

The initial grid is wider and is different for each source-family type. The loop grid explores OpenMP schedules (`static`, `dynamic`, `guided`), chunk sizes, partition/join thread counts and partition block sizes. The explicit-task grid explores the number of input blocks per partition task, the number of partitions per join task, offset-task granularity and thread counts. The taskloop grid explores the same decomposition through `taskloop` grainsizes. Each family has its own statistics notebook/file (`omp_loop_statistics.ipynb`, `omp_task_statistics.ipynb` and `omp_taskloop_statistics.ipynb`) used to summarize the larger grid and select the reduced one.

The reduced grid used for the final results keeps the most stable choices found by those statistics files: guided scheduling for loop variants, explicit task grains in the range 2–4 for the task variants, taskloop grains in the range 2–8 for taskloop variants, 32 join threads for most best points, and 32 or 64 partition threads depending on the implementation.

Three datasets are measured. The uniform dataset is the same pseudo-random generation scheme used previously. The skewed datasets are generated with the format `skewed_R_H`: $R\%$ of records are sampled from keys that map to the first $H\%$ of partitions. In the final campaign I used `skewed_90_5` and `skewed_90_1`, which concentrate 90% of the records respectively into about 13 and 3 hot partitions.

2 Parallelization Strategy

The independent partition structure remains the main source of parallelism. The difference with respect to the C++ threaded version is that thread management, loop scheduling and task queues are delegated to OpenMP.

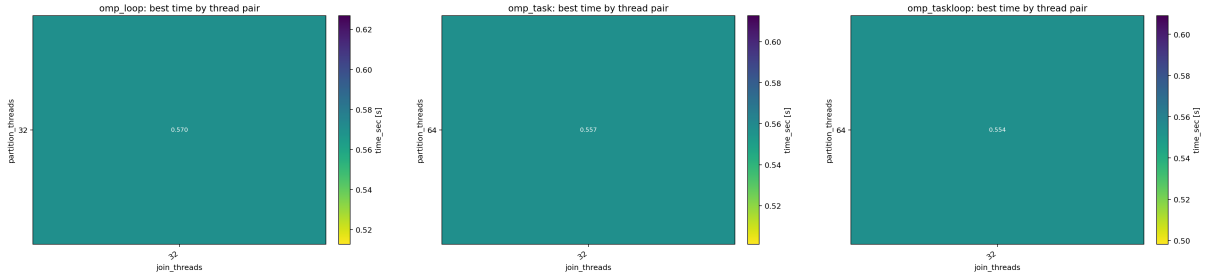

Figure 1: Thread-count sensitivity from the large grid search for loop, explicit-task and taskloop OpenMP implementations.

2.1 Partition phase

All OpenMP variants use a blocked partitioning pipeline. Each relation is split into input blocks; the histogram step maps keys to partition ids and fills block-local histograms. The global partition starts are then computed with an exclusive prefix sum over the P partition counters, and a second parallel step scatters records into contiguous partition ranges using precomputed block offsets. The expensive loops are therefore the histogram and scatter loops; the prefix scan remains small because $P = 256$.

In `hashjoin_omp_loop.cpp` and `hashjoin_omp_loop_wb.cpp`, histogram and scatter are expressed with `omp for schedule(runtime)`. This makes the same executable usable with `static`, `dynamic` and `guided` policies. In `hashjoin_omp_task.cpp`, explicit tasks are created over batches of input blocks and offset partitions. In `hashjoin_omp_taskloop.cpp`, the same decomposition is expressed with `omp taskloop grainsize(...)`.

2.2 Join phase

After partitioning, each partition can be joined independently. The loop implementation uses an OpenMP reduction over `join_count`, `checksum1` and `checksum2`. The explicit-task implementation writes each partition result into a private slot of a `partial` array and reduces it sequentially after `taskwait`. The taskloop implementation follows the same partial-result scheme, but lets OpenMP create the tasks from a `taskloop`.

The `_wb` variants address skew by ordering partitions by estimated work,

$$|R_p| + |S_p|,$$

from largest to smallest before scheduling the local joins. The loop `_wb` version then uses `schedule(dynamic,1)` on this ordered list; the task and taskloop `_wb` versions create tasks over the ordered list. This is the main difference under skewed datasets: without this reordering, a few hot partitions can dominate the total join time even if many threads are available.

2.3 Scaling files

`strong_scaling.cpp` and `weak_scaling.cpp` are dedicated benchmark executables. They use the workload-balanced loop strategy with guided partition scheduling and dynamic join scheduling, because this was the most stable choice across datasets. Strong scaling fixes $N_R = N_S =$

50 000 000 and varies the thread count. Weak scaling increases the input from 10M to 80M records per relation while increasing the thread count from 4 to 32.

2.4 OpenMP tuning

The statistics plots confirm the reason for reducing the benchmark grid. Loop scheduling is mostly sensitive to the partition/join thread combination and to schedule choice, while task-based implementations are more sensitive to task granularity. Very fine grains create too many tasks; very coarse grains reduce balancing opportunities. This is why the final grid keeps only the smaller grain values that were consistently close to the best regions.

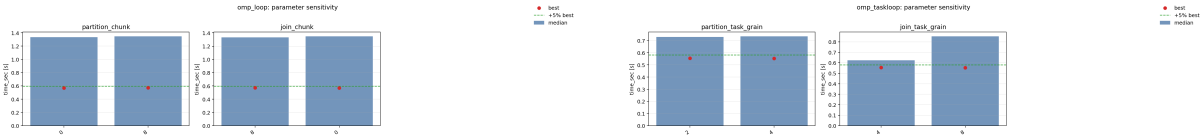

Figure 2: Examples of parameter sensitivity used to reduce the final benchmark grid.

3 Results Analysis

All results below are generated from `src/results/`. The best sequential times in the current campaign are 3.114 s for the uniform dataset, 3.258 s for `skewed_90_5`, and 3.256 s for `skewed_90_1`. The best Module 2 C++ threaded result available in `src/old_results/` was 0.507 s for `hashjoin_par_pj_wb_map`; the best broad-grid OpenMP result here is 0.539 s, so the final OpenMP implementation is close in absolute time but does not clearly improve over the previous C++ threaded best case.

3.1 Uniform dataset

On the uniform workload, all complete OpenMP implementations are close once both partition and join are parallelized. The best time is obtained by `hashjoin_omp_taskloop_wb` with 64 partition threads and 32 join threads: 0.539 s, corresponding to 5.78× speedup over the current sequential baseline. The difference between the best loop, task and taskloop versions is small; this means that, for uniform partitions, OpenMP tasking does not add much by itself. The dominant gains come from parallelizing the join and from keeping the local hash table cache-friendly.

Variant	T_{part}	T_{join}	Best time [s]	Speedup
`omp_loop`	32	32	0.568	5.49×
`omp_loop_wb`	32	32	0.561	5.55×
`omp_task`	64	32	0.545	5.72×
`omp_task_wb`	64	32	0.542	5.74×
`omp_taskloop`	64	32	0.544	5.73×
`omp_taskloop_wb`	64	32	0.539	5.78×

Table 1: Best observed uniform-dataset configuration per OpenMP executable.

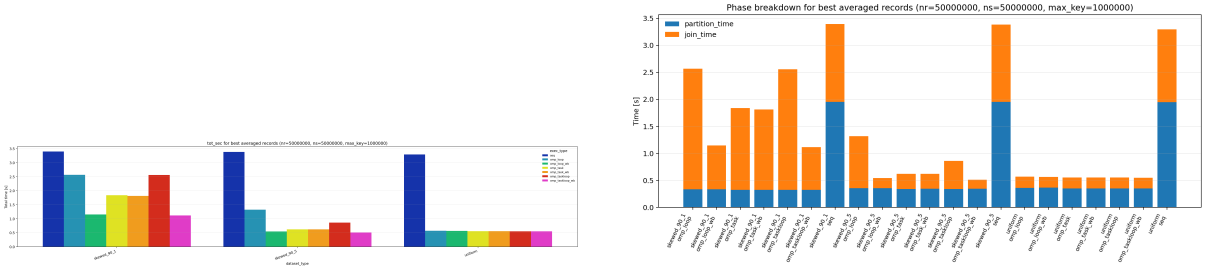

Figure 3: Best total time and phase breakdown for the current OpenMP campaign.

3.2 Skewed datasets

The skewed datasets change both load balance and the actual join workload. The final match count rises from about $2.50 \cdot 10^9$ on uniform data to $4.06 \cdot 10^{10}$ on `skewed_90_5` and $1.74 \cdot 10^{11}$ on `skewed_90_1`. Therefore the slowdown is not only scheduling overhead: the hot partitions also contain many more duplicate matches.

Dataset	Best OpenMP variant	Best time [s]	Join time [s]	Speedup
`uniform`	`taskloop_wb`	0.539	0.191	5.78×
`skewed_90_5`	`taskloop_wb`	0.501	0.163	6.51×
`skewed_90_1`	`taskloop_wb`	1.101	0.781	2.96×

Table 2: Best OpenMP result for each dataset, using the corresponding sequential baseline.

The workload-balanced versions are essential under skew. For `skewed_90_5`, the loop version improves from 1.281 s to 0.525 s when workload balancing is enabled. For `skewed_90_1`, the same change improves from 2.558 s to 1.120 s. The strongest skew still remains slower because a very small number of partitions becomes too large to be split further by the current partition-level join strategy.

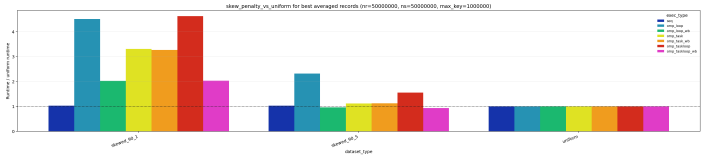

Figure 4: Penalty of the skewed datasets with respect to the uniform dataset.

3.3 Strong and weak scaling

Strong scaling is measured with the dedicated workload-balanced loop executable. On the uniform dataset, speedup grows up to 6.34× at 32 threads and then slightly decreases at 64 threads. The `skewed_90_5` dataset follows the same shape but with a lower plateau, while `skewed_90_1` saturates very early around 2.9×. This confirms that the current granularity is the partition: if one hot partition dominates, adding more threads cannot split that single local join.

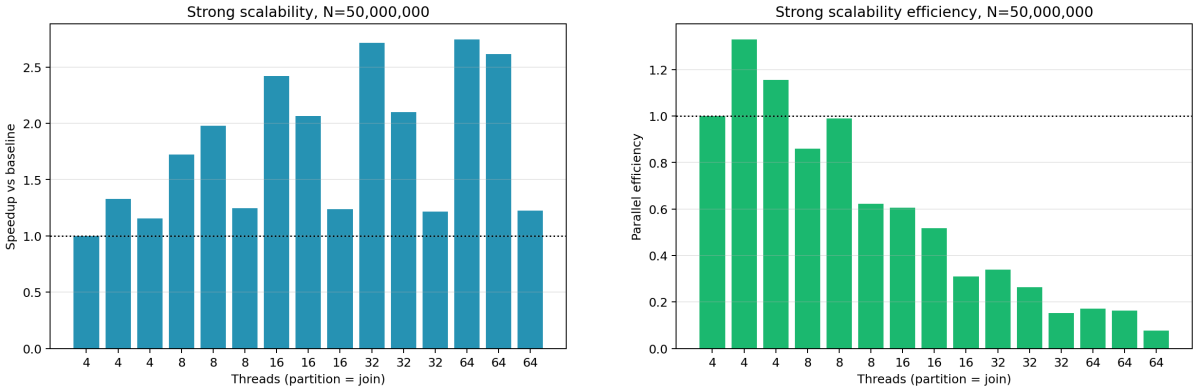

Figure 5: Strong-scaling speedup and efficiency for the dedicated workload-balanced OpenMP executable.

Weak scaling is moderate. Uniform data keeps the best scaled speedup, reaching about 2.20× at 32 threads relative to the 4-thread base point, but efficiency still drops to about 0.28. The `skewed_90_1` curve is much worse and reaches only 0.89× scaled speedup at 32 threads, because the enlarged hot partitions increase both imbalance and duplicate-match work.

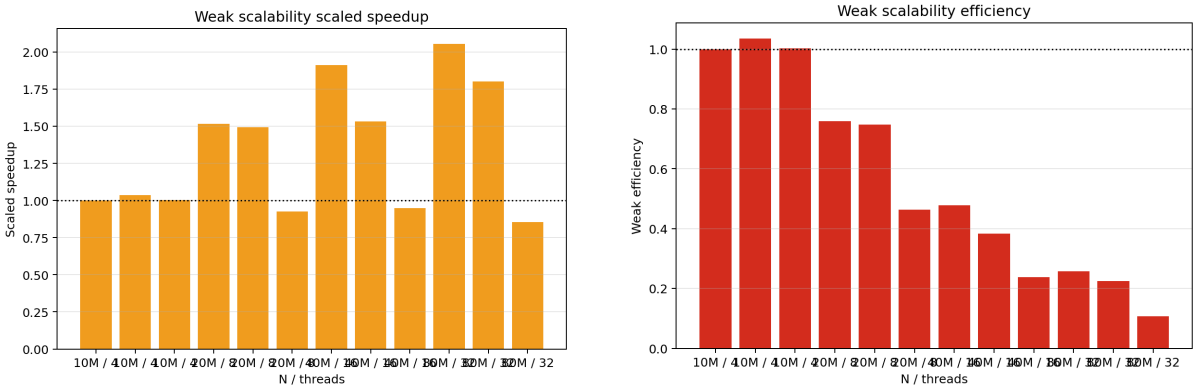

Figure 6: Weak-scaling scaled speedup and efficiency for uniform and skewed datasets.

4 Correctness Validation

Correctness is checked at two levels. For small inputs, each executable can run the naive $O(N_R N_S)$ verifier and print `naive_join_count`, `naive_checksum1` and `naive_checksum2`. For benchmark-size inputs, `checker.cpp` scans the Slurm output files and compares `join_count`, `checksum1` and `checksum2` for each executable and dataset type. The checker is outside the measured region, and the collected runs match across implementations for the same dataset.

5 Conclusions

The OpenMP implementations preserve the previous algorithm and make the scheduling trade-offs clearer. On uniform data, loop, task and taskloop variants perform similarly once the join phase is parallelized; tasking does not provide a large advantage because partitions are already well balanced. On skewed data, workload-aware join ordering is the decisive change, especially for `skewed_90_5`. The strongest skew still exposes a limitation of partition-level parallelism: one hot partition cannot be split among multiple threads by the current local join design.

Compared with the best C++ threaded result from Module 2, the best OpenMP result is close but slightly slower in the broad campaign. The main benefit of this module is therefore not a new absolute minimum, but a clearer understanding of which OpenMP constructs are sufficient: guided loop scheduling plus workload-aware dynamic join scheduling is the most robust choice for this implementation.